

SC20001 : Lab 1

Hybrid Sort

Presented by

- Raj Yash
- Ramachandran Saichandar
- Rane Shruti Devendra



Why Hybridize Merge Sort?



Merge Sort

- Divide the array into two halves
- Recursively sort both halves
- Merge the sorted halves
- Time complexity: $\Theta(n \log n)$
- Efficient and predictable for large datasets
- Weakness: recursion + merging small subarrays adds overhead



Insertion Sort

- Build a sorted portion one element at a time
- Insert each new element into its correct position
- Time complexity: $\Theta(n^2)$
- Simple control flow with low overhead
- Strength: efficient for small subarrays

Hybrid idea

Merge Sort for large subarrays → Insertion Sort for small subarrays

Threshold S : What it does

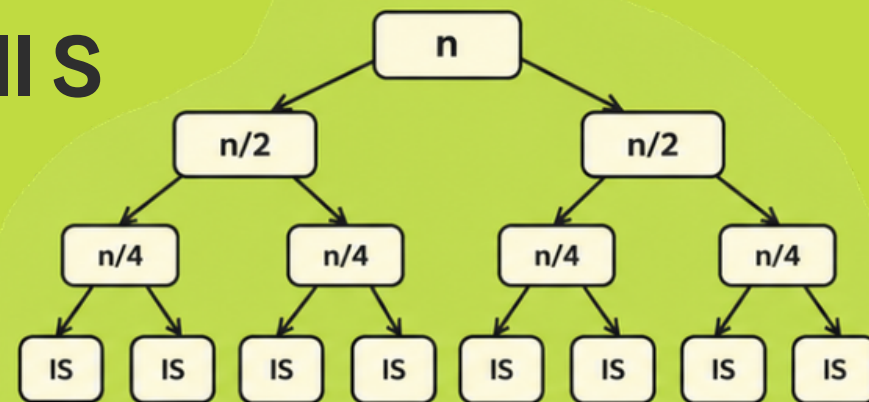
Hybrid rule

If subarray size $> S \rightarrow$ continue Merge Sort | If subarray size $\leq S \rightarrow$ switch to Insertion Sort



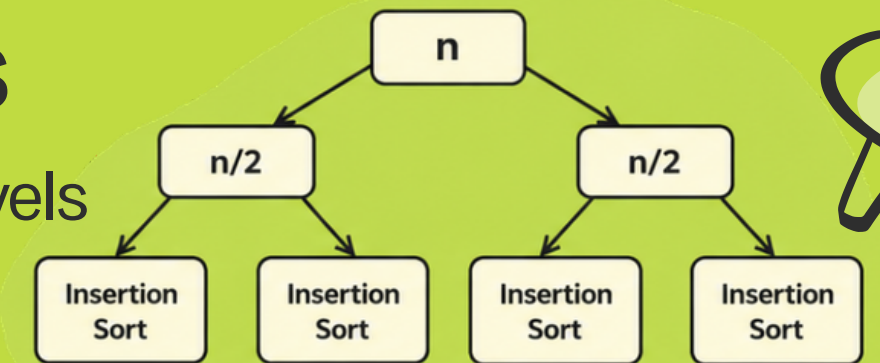
Effect of small S

- More recursive levels
- More merge operations
- Smaller insertion sort leaves



Effect of large S

- Fewer recursive levels
- Larger insertion sort leaves
- More insertion-sort comparisons



S controls the trade-off between merge recursion depth and insertion sort leaf size

Hybrid Algorithm Code

```
def hybrid_merge_sort(values: list[int], threshold: int) -> int:
    if threshold < 1:
        raise ValueError("threshold must be at least 1")
    auxiliary = [0] * len(values)

    def sort(left: int, right: int) -> int:
        size = right - left + 1
        if size <= threshold:
            return insertion_sort_range(values, left, right)
        mid = (left + right) // 2
        comparisons = sort(left, mid)
        comparisons += sort(mid + 1, right)
        comparisons += merge(values, auxiliary, left, mid, right)
        return comparisons

    return 0 if not values else sort(0, len(values) - 1)
```



Merge Sort

```
def merge_sort(values: list[int]) -> int:
    auxiliary = [0] * len(values)

    def sort(left: int, right: int) -> int:
        if left >= right:
            return 0
        mid = (left + right) // 2
        comparisons = sort(left, mid)
        comparisons += sort(mid + 1, right)
        comparisons += merge(values, auxiliary, left, mid, right)
        return comparisons

    return sort(0, len(values) - 1)
```

Efficiently merges large, sorted
subarrays in $O(n)$ time

Insertion Sort

```
def insertion_sort_range(values: list[int],
                          left: int, right: int) -> int:
    comparisons = 0
    for i in range(left + 1, right + 1):
        key = values[i]
        j = i - 1
        while j >= left:
            comparisons += 1
            if values[j] <= key:
                break
            values[j + 1] = values[j]
            j -= 1
        values[j + 1] = key
    return comparisons
```

Efficiently sorts small subarrays
of size S

Input Generation and Timing Setup



Dataset generation

- Random seed = 2001
- Values sampled from [1, 10,000,000]
- Input sizes include $n = 1,000$ up to 10,000,000
- Same unsorted dataset reused within each controlled comparison

Timing

- `time.process_time_ns()`
- Copy input before timing
- Time only the sorting call
- Validate sorted output after timing

- **Process CPU time (not clock time)**
- **Copying & Validation excluded from timing**



Theoretical Model

Merge work above cutoff

$$\Theta\left(n \log_2 \frac{n}{S}\right)$$

Insertion Sort on leaf blocks

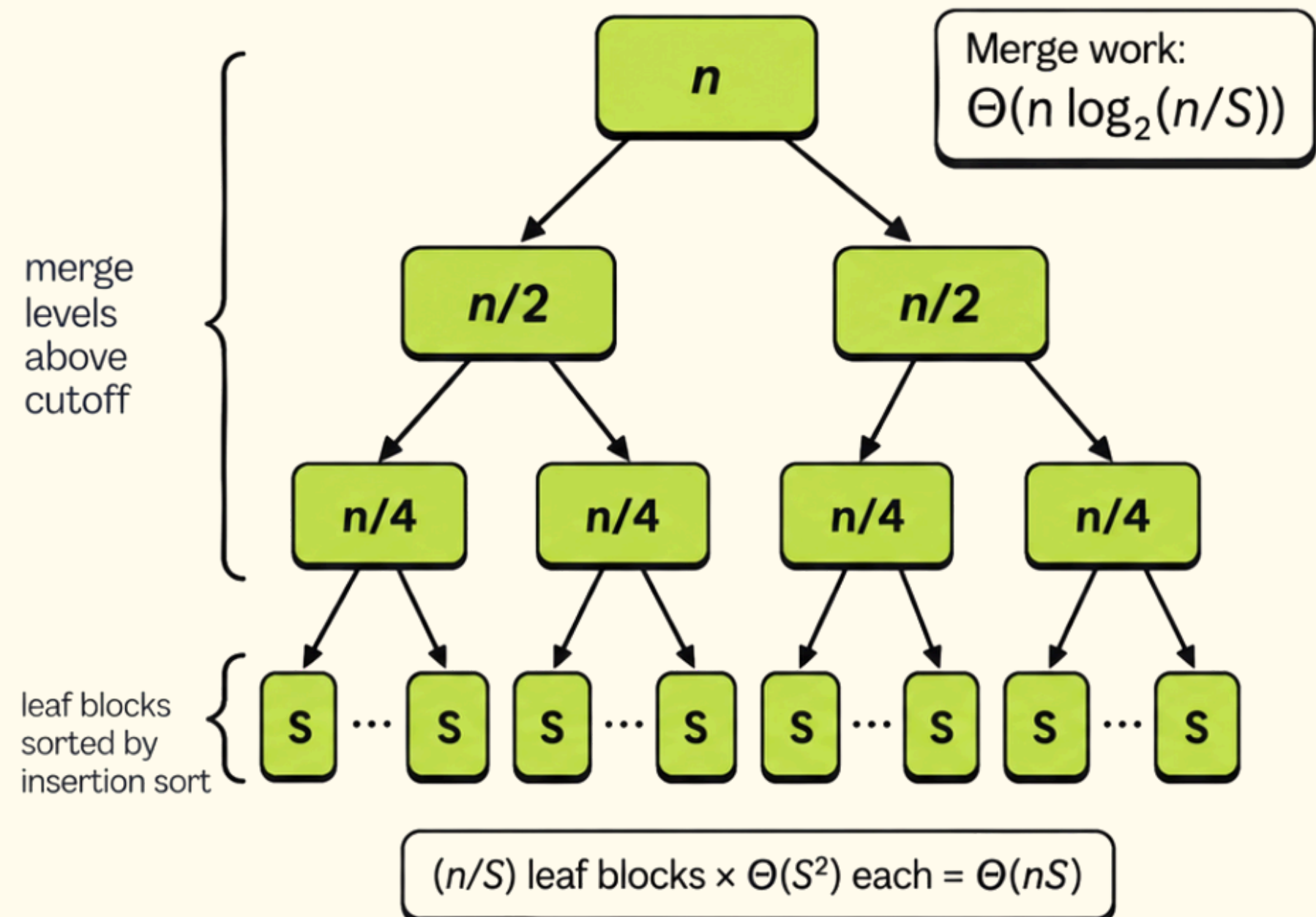
$$\frac{n}{S} \cdot \Theta(S^2) = \Theta(nS)$$

Reference model

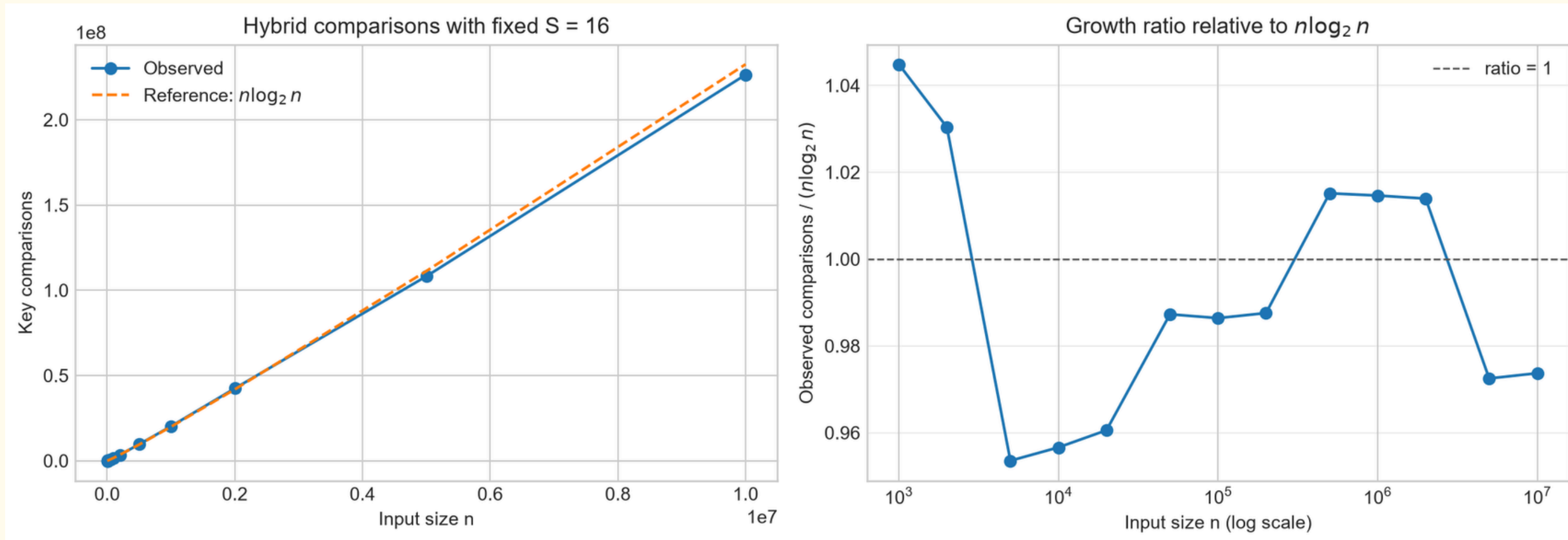
$$T(n, S) = \Theta\left(n \log_2 \frac{n}{S} + nS\right)$$

If S is fixed

$$T(n, S) = \Theta(n \log n)$$



C(i): Fixed S, Varying Input Size n



Theoretical expectation

$$T(n, 16) = \Theta(n \log n)$$

For fixed S, we therefore expect

$$C(n) \propto n \log_2 n$$

Empirical observation

- $\frac{n \log_2 n}{C(n)}$ remains roughly 0.95–1.04
- A roughly stable normalized ratio supports proportional $n \log_2 n$ growth

C(ii): Fixed n , Varying Threshold S

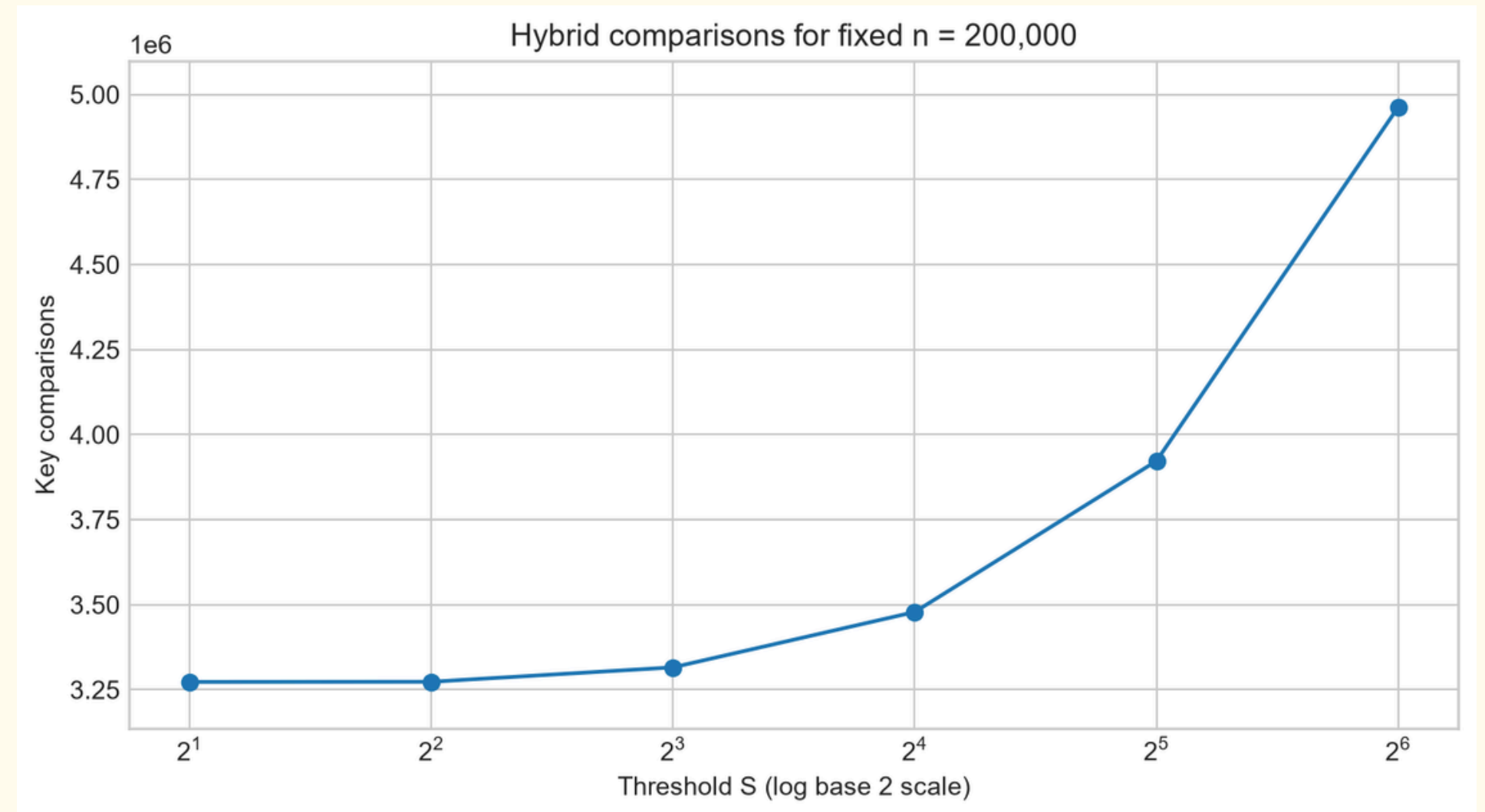
Results

$S=64$ performs
~51.6% more
comparisons
than $S=2$

	n	S	comparisons
0	200000	2	3272851
1	200000	4	3273283
2	200000	8	3315694
3	200000	16	3478396
4	200000	32	3922280
5	200000	64	4962322

Interpretation

- Small $S \rightarrow$ more Merge Sort recursion, but small Insertion Sort leaves
- Large $S \rightarrow$ fewer recursive levels, but larger Insertion Sort leaves
- As S becomes large, the extra Insertion Sort work dominates the comparison count



For key comparisons alone, the best tested threshold was $S=2$

C(iii) Setup & Finding a Practical S

C(iii) experiment

- Input sizes: 100,000; 500,000; 1,000,000; 5,000,000
- Tested S: 2, 4, 8, 12, 16, 20, 24, 32, 48, 64
- 5 CPU-time runs per (n, S)
- Median CPU time used for stability and reduce sensitivity to noisy single timing runs

A) based on key comparisons

Table C5: S with the fewest exact comparisons for each input size

	n	comparison_best_S	comparisons
0	100000	2	1536133
10	500000	2	8837346
20	1000000	2	18675172
30	5000000	2	105050548

- Least number of comparisons occur at $S = 2$ because the insertion-sort blocks are kept very small, minimizing its $O(S^2)$ comparison cost
- Comparison count increases as S grows because insertion-sort blocks become larger

C(iii) Finding the Practical S

B) based on CPU time

How we selected practical threshold S

- For each n, normalize each median runtime by the fastest measured runtime for that n
- Average the relative runtime across all four input sizes
- Select the S with the lowest average

Table C4: CPU-time penalty by S (0% is the fastest measured time for that n)

	n=100,000	n=500,000	n=1,000,000	n=5,000,000
S				
2	28.6%	15.9%	11.3%	10.4%
4	14.3%	4.5%	4.1%	6.2%
8	14.3%	0.0%	0.0%	2.6%
12	14.3%	0.0%	1.0%	0.0%
16	0.0%	0.0%	0.0%	0.0%
20	14.3%	0.0%	0.0%	1.4%
24	0.0%	0.0%	0.0%	1.2%
32	14.3%	6.8%	6.2%	1.4%
48	14.3%	4.5%	5.2%	8.4%
64	28.6%	27.3%	25.8%	8.1%

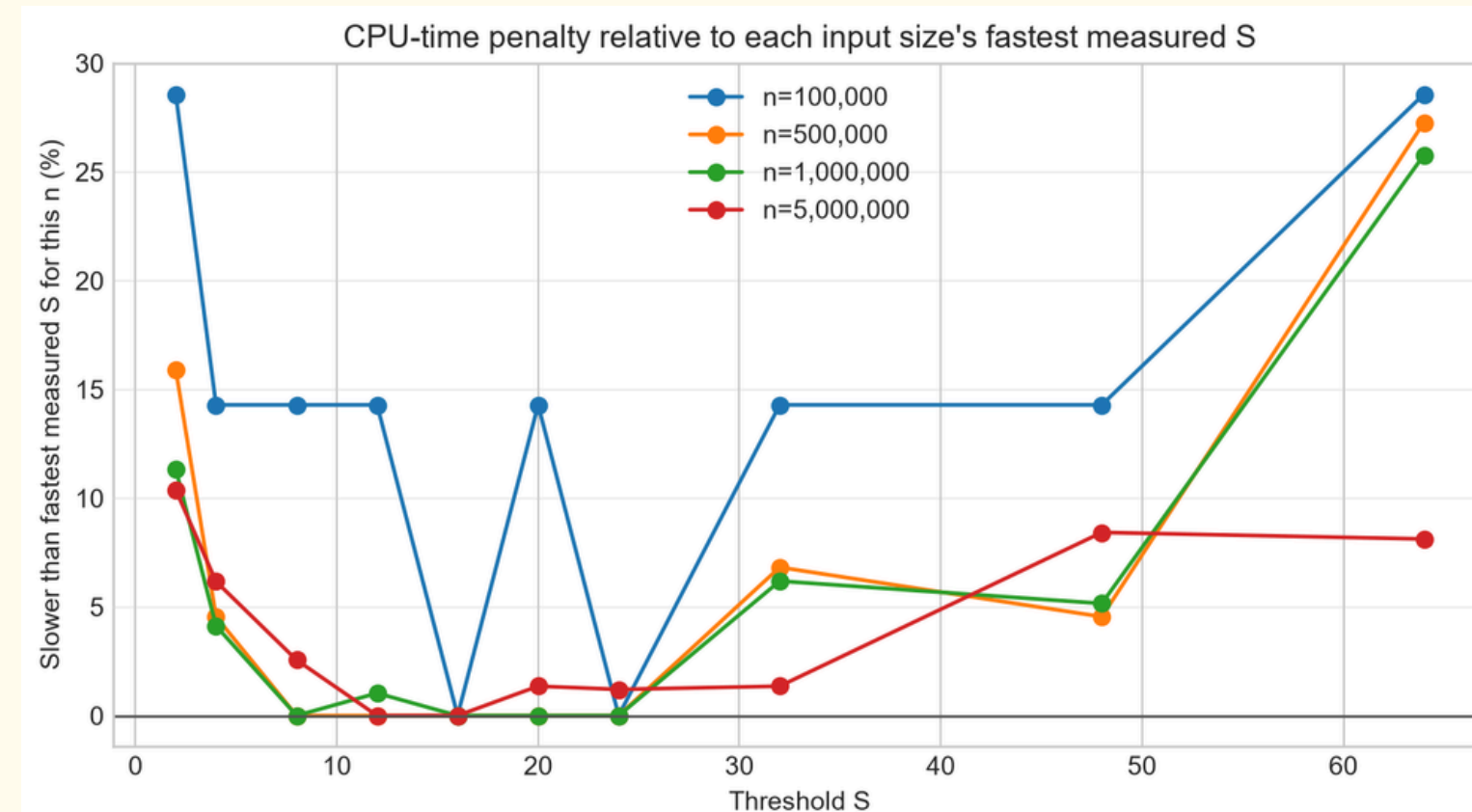


Table C6: S with the fastest measured median CPU time for each input size

	n	time_best_S	median_cpu_time
4	100000	16	0.109375
12	500000	8	0.687500
22	1000000	8	1.515625
33	5000000	12	10.390625

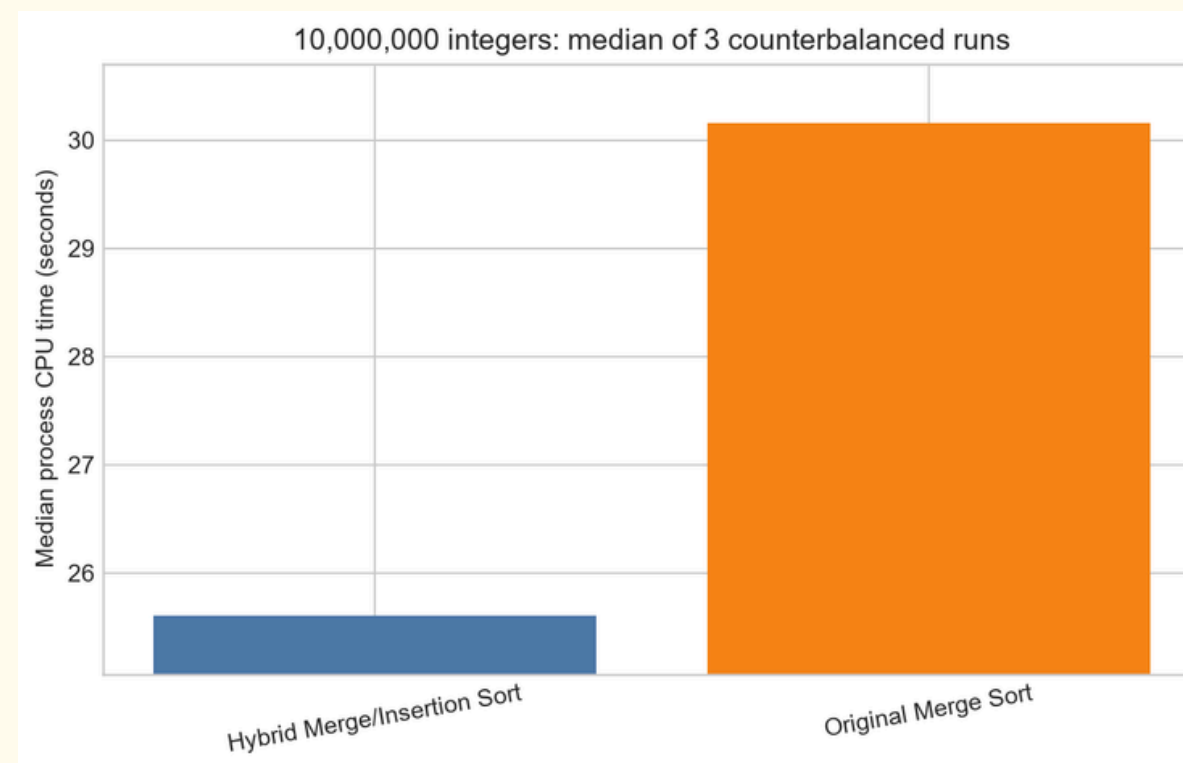
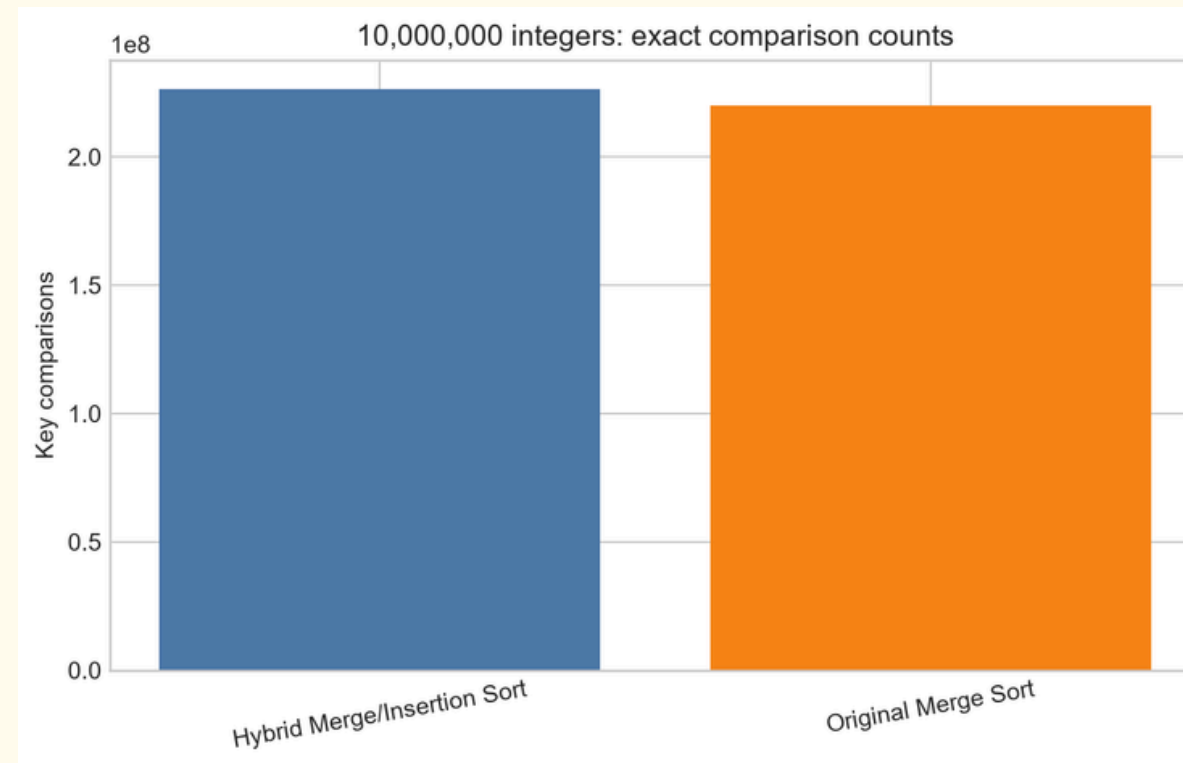
Part D: Original Merge vs Hybrid Sort

Part D setup

- $n = 10,000,000$
- Hybrid threshold: $S = 16$
- 3 timing runs per algorithm
- Counterbalanced order:
 - Merge $\rightarrow$ Hybrid $\rightarrow$ Hybrid $\rightarrow$ Merge $\rightarrow$ Merge $\rightarrow$ Hybrid
 - Done to reduce run-order bias

Result

Algorithm	Comparisons	Median CPU Time (sec)
Hybrid Sort	226415036	25.609375
Original Merge Sort	220098332	30.15625



Key differences

- Hybrid: +2.870% key comparisons
- Hybrid: -15.078% CPU time
- Speedup: 1.178×

More comparisons did not mean slower execution: Hybrid Sort was ~15.1% faster in our experiment

Conclusion

- Hybrid Merge/Insertion Sort preserves the expected $\Theta(n \log n)$ behaviour when the threshold S is fixed.
- Our experiments also showed the expected trade-off: increasing S reduces recursive Merge Sort work, but causes Insertion Sort to handle larger subarrays and perform more key comparisons.
- For practical performance, minimum comparisons did not correspond to minimum CPU time. Using results across multiple input sizes, we selected $S=16$ as the best aggregate measured threshold for our implementation.
- At $n=10,000,000$, the Hybrid Sort used slightly more comparisons but achieved lower median CPU time than standard Merge Sort.

Overall, the hybrid approach improved practical runtime by reducing recursion and merge overhead on small subarrays, rather than by minimizing the number of comparisons.

